

12장 트러블슈팅

12.1 개요

개발을 진행하면서 문제가 생긴 구간을 파악하고, 재발 방지 및 학습하기 위하여 트러블슈팅을 작성하였다.
팀원별로 맡은 도메인 · 개발 범위가 각자 다르기에 팀원별로 내용을 작성한다.

최재용

12.2.1 수집 · 연동

- 사례 1. DB에 없는 행에 알람 발화
- 원인 : bulk_create(ignore_conflicts) 후 원본 objs로 트리거
 - 해결 : UNIQUE로 saved 행 재조화 → 그 행만 trigger
 - 교훈 : 쓰기 입력(objs) ≠ 쓰기 결과(저장된 행). 부분 성공 가능한 연산 뒤엔 DB가 확정된 행을 다시 읽어 후속 처리한다

- 사례 2. 펌웨어 -1 (통신불능 상태) 를 0 (유효값) 과 혼동
- 원인 : -1이 이상치로 오해됨 (설계와 다름)
 - 해결 : value=NULL + sensor_status 분리
 - 교훈 : 센서에서 0은 유효값, 결측/오류는 값이 아니라 상태 컬럼으로 설정. 매직넘버를 섞으면 데이터가 오염된다.

12.2.2 AI 추론

- 사례 1. 1틱 스파이크 즉시 DANGER
- 원인 : 1틱으로 지속성을 판단
 - 해결 : watt 측 danger 2틱 confirm (위험의 실시간성은 살리며, 중복 최소화)
 - 교훈 : 순간값 한 틱으로 위험 확정 금지. 지속성(연속 N틱) 게이트를 두되 축별 의미가 달라 적용 범위 선정 필요 (추가 모니터링 필요)

- 사례 2. ARIMA 단발 스파이크 미포착
- 원인 : ARIMA(1,1,1)이 spike추종, CI 미위반
 - 해결 : 의도된 한계 (아리마는 이상수치가 아닌 이상패턴 탐지) + IF 결합 보완
 - 교훈 : 미탐이 전부 버그는 아니다. 기능이 가능한 것과 불가능한 것을 파악하고, 보완해야 한다

12.2.3 알람 중복

- 사례 1. 전력 알람 폭주 (해소 완료 후 재발화)
- 원인 : 16 채널이 Event 1개 공유, 한 채널이 복귀되어 해소시 새 Event_id 생성 및 중복알람 발생
 - 해결 : 마지막 활성 채널만 RESOLVE 되었을 경우 해소
 - 교훈 : 다대일에서 '하나의 복귀'를 '전체 해제'로 처리하면 상태가 churn하며 dedup이 무너진다. 집합의 마지막 원소까지 확인하고 해제

```
if has_other_active_channel(device_id, channel, channel_count):
    # 다른 채널 위험 지속 + Event 유지 (마지막 채널이 달는다). 폭주 차단 지점.
    logger.info(
        "전력 정상화 알림 | device=%s ch=%s - 다른 채널 위험 지속, Event 유지",
        device_id,
        channel,
    )
else:
    resolved = auto_resolve_active_events(
        event_type_prefix="power", power_device_id=device_id
    )
```

<동작 증명>

```
keys = [
    _state_key(device_id, ch)
    for ch in range(1, channel_count + 1)
    if ch != exclude_channel
]
if not keys:
    return False
states = cache.get_many(keys)
return any(s in (RiskLevel.WARNING, RiskLevel.DANGER) for s in states.values())
```

<동작 증명>

이성현

12.2.4 인프라

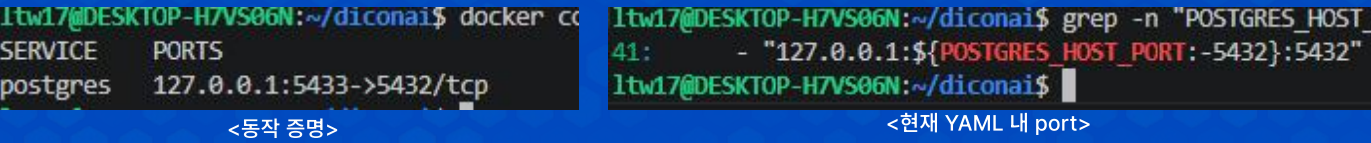
- Docker 포트 5432 충돌 (WSL2 환경).
- 증상: docker compose up -d 시 ports are not available: exposing port TCP 127.0.0.1:5432 에러로 postgres 기동 실패.

- 원인 : Windows-WSL2 어디에도 5432를 쓰는 프로세스가 없는데 점유 상태만 남아 있었음. /proc/net/tcp로 추적해 보니 어떤 프로세스에도 속하지 않은 소켓이 남아 있었고, Docker Desktop 내부 프록시(vpnkit)가 이전 세션의 포트 상태를 정리하지 못한 채 잔류한 것이 원인이었음.

- 시도와 실패 down 후 재시도, wsl --shutdown, Docker Desktop 강제 종료, override 파일 작성을 시도했으나 모두 실패. 특히 override는 Compose의 ports가 MERGE(append) 방식이라 원본 5432가 그대로 남아 효과가 없었음.

- 해결 : 호스트 포트를 환경변수로 분리
- 변경 전 - "127.0.0.1:5432:5432"
- 변경 후 - "127.0.0.1:\${POSTGRES_HOST_PORT:-5432}:5432"

포트를 직접 5433으로 박는 대신 환경변수(POSTGRES_HOST_PORT, 기본값 5432) 로 분리했음.
충돌이 나는 사람만 자신의 .env에서 “5433” 등으로 바꾸면 되고, 컨테이너 내부 통신은 postgres:5432로 그대로라 앱 동작에 는 영향이 없음.



<동작 증명>

<현재 YAML 내 port>

정휘훈

12.2.5 모니터링

사례 1. Prometheus /metrics 500 에러

- 원인 : 컨테이너가 비정상 종료된 후 재시작할 때 이전 프로세스가 PROMETHEUS_MULTIPROC_DIR에 남긴 stale .db 파일이 정리되지 않았다. MultiProcessCollector가 손상된 파일을 읽으면서 JSON 파싱 실패가 발생해 /metrics 엔드포인트 전체가 500을 반환했고, Grafana 패널 전체가 공백 상태가 됐다.

- 해결 : entrypoint.sh에 서버 기동 전 find ... -name "*.db" -delete 한 줄을 추가했다. 컨테이너가 재시작될 때마다 항상 깨끗한 상태에서 메트릭 수집이 시작된다.

사례 2. Prometheus multiproc PID 충돌 — 메트릭 한쪽만 표시

- 원인 : DRF 컨테이너와 Celery 컨테이너는 각자 독립된 PID 네임스페이스에서 PID 1부터 시작하기 때문에 동일한 파일명(gauge_max_1.db)을 생성한다. 두 MultiProcessCollector를 같은 registry에 등록하면 동일 메트릭이 중복 출력되고, Prometheus는 첫 번째 값만 취해 나머지 컨테이너의 수치가 완전히 무시됐다.

- 해결 : 수집 시점에 임시 디렉토리를 생성하고 컨테이너별로 PID 오프셋을 적용해 파일명을 변환한 뒤 복사했다. DRF는 오프셋 0, Celery는 오프셋 100,000을 적용했으며, 컨테이너 PID 최대값(65,535)보다 큰 오프셋이라 두 범위가 절대 겹치지 않는다.

